

Hybrid Potential in Combinatorial Optimization: Lessons from the N-Queens Benchmark

Freeha Saeed
Dept. of Computer & Software Engg.
College of EME, NUST
Rawalpindi 46000, Pakistan
fareeha.bsee319new@gmail.com

Kimia Sadat Karbasi
Department of Business
Univ. of Europe for Applied Sciences
Potsdam 14469, Germany
kimia.karbasi@ue-germany.de

Iqra Suleman
Dept. of Computer & Software Engg.
College of EME, NUST
Rawalpindi 46000, Pakistan
iqrasuleman3435@gmail.com

Raja Hashim Ali
Department of Artificial Intelligence
GIK Institute of Engg. Sciences & Tech.
Topi 23460, Pakistan.
hashim.ali@giki.edu.pk

Muhammad Zeeshan Khan
Department of Cyber Security
GIK Inst. of Engg. Sciences & Technology
Topi 23460, Pakistan
u2022644@giki.edu.pk

Muhammad Imad
School of Management
Univ. of Comp. & Emerging Sciences
Islamabad 44000, Pakistan.
muhammad.imad@isb.nu.edu.pk

Qamar Abbas
Department of Computer Science
GIK Institute of Engg. Sciences & Tech.
Topi 23460, Pakistan.
qamar.abbas@giki.edu.pk

Muhammad Umar Maqsood
Department of Cyber Security
GIK Institute of Engg. Sciences & Tech.
Topi 23460, Pakistan.
u2022447@giki.edu.pk

Debopriya Das
Department of Business
Univ. of Europe for Applied Science
14469 Potsdam, Germany
debopriya.das@ue-germany.de

Abstract—The N-Queens problem is a classical benchmark in artificial intelligence and combinatorial optimization, requiring the placement of N queens on an $N \times N$ chessboard without conflicts. This study presents a comparative evaluation of four algorithmic strategies: Exhaustive Search using depth-first backtracking, Greedy Search with hill climbing, Simulated Annealing, and Genetic Algorithms. Each method was implemented in Python and tested under identical hardware conditions on board sizes $N = 10, 30, 50, 100$, and 200 . Performance was assessed in terms of solution success, runtime, memory usage, and scalability. Experimental results show that Exhaustive Search performs reliably for small instances ($N < 10$) but becomes infeasible for larger boards due to exponential growth in time and memory. Greedy Search demonstrates efficiency up to $N = 50$ but frequently stagnates in local optima at higher N . Simulated Annealing achieves a strong balance, solving instances up to $N = 100$ with moderate computational cost. Genetic Algorithms also solve $N = 100$ but incur significantly higher memory and runtime overheads. No single approach proved universally optimal, emphasizing the importance of parameter tuning. The findings suggest that adaptive or hybrid algorithms may provide improved scalability for complex combinatorial problems.

Index Terms—N-Queens, Depth First Search, Greedy Search, Simulated Annealing, Hill Climbing, Genetic Algorithm, Memory usage, Time, Performance, Scalability

I. INTRODUCTION

In several recent times, recent comparative research has examined both classical and stochastic optimization techniques for solving the N-Queens problem across different board sizes

[1]–[3]. On one hand, accuracy is central to modern artificial intelligence based systems, and it is an important metric for deployment in real-life practical systems. On the other hand, stochastic and evolutionary algorithms provide alternative strategies by exploring the solution space in a probabilistic manner, which then automatically helps by reducing the risk of getting trapped in local optima [4], [5]. For these reasons among other reasons, the N-Queens problem continues to serve as an important experimental platform for analyzing and comparing different algorithmic paradigms in artificial intelligence and combinatorial optimization.

The N-Queens formulation is not simply just a theoretical textbook problem - it reflects the structure of many real-world constraint satisfaction and resource allocation problems. In fact, several applications, for example graph coloring, scheduling, and resource placement, share similar combinatorial characteristics to N-Queens, which makes the N-Queens benchmark useful for evaluating algorithmic strategies [6]. At the same time, solution search techniques involving various metaheuristic approaches, for example, genetic algorithms and evolutionary optimization, have been widely applied to related optimization problems due to their ability to efficiently search large solution spaces [7]. Large-scale optimization systems in modern computing environments further require algorithms that can balance solution quality with computational cost and scalability [8]. Recent research conducted in the fields of Computer Vision and Natural Language Processing, with several machine learning and artificial intelligence applications, has demonstrated the effectiveness of optimization-

based techniques in solving complex decision problems [9], [10]. Similarly, algorithmic improvements in AI-based systems, especially for Natural Language Processing and Large Language Models, have highlighted the importance of systematic experimentation and performance benchmarking across multiple configurations [11], [12]. These recent developments have motivated the continued investigation of algorithmic performance on canonical problems, such as N-Queens, to better understand the scalability as well as the robustness of various search strategies. Large-scale data analytics and computational modeling frameworks have been increasingly used to analyze patterns and improve predictive performance in scientific and engineering applications [13], [14]. Furthermore, the integration of explainable artificial intelligence techniques has been popular recently for improving the transparency and interpretability of algorithmic decision processes in complex computational systems [15]–[17]. These developments highlight the importance of systematic experimental evaluation of algorithmic approaches, particularly in benchmark optimization problems, especially for the N-Queens problem.

A. Research Questions

We have addressed the following main research questions in this study.

- 1) Is exhaustive search able to solve large-scale instances of the N-Queens problem?
- 2) What does the hill-climbing algorithm's tendency to get trapped in local optima has on its ability to find answers to the N-Queens problem when N gets higher?
- 3) Can simulated annealing break through local optima to solve the N-Queens problem as N rises, and what important parameters exist in the process?
- 4) How well the N-Queens problem is solved using genetic algorithms by changing the parameters and combining them with other techniques?

B. Novelty of this study

This main aim of this study is to design and implement four different algorithms, namely Exhaustive Search, Greedy Search, Simulated Annealing, and Genetic Algorithm, for solving the N-Queens problem. We have assessed how well-organized each method is with memory and speed usage. Additionally, new parameter settings and optimized techniques have been explored and assessed to solve this problem. Also, this study covers, compares, and analyzes these strategies to understand and explore which of these strategies performs better under various circumstances.

II. LITERATURE REVIEW AND CONTRIBUTION

We tried various studies and found that the classical search-based approaches have been widely studied for their correctness guarantees. However, these studies are often limited by computational complexity for larger instances but are extremely accurate for smaller instances of input, in this case, the size N of the problem [5]. To address this particular limitation, we found that several researchers have explored stochastic

and evolutionary techniques, where these techniques use probabilistic exploration to improve convergence rates [2], [3]. Comparative evaluations of hill climbing, simulated annealing, and genetic algorithms in many studies have demonstrated the notable differences in runtime efficiency and success rates across problem scales with the size of the input N [1], [4]. Related recent work in machine learning and combinatorial optimization has shown to us that evolutionary algorithms can effectively solve complex routing and scheduling tasks when appropriate operators and parameter configurations are employed [7]. Optimization research, therefore, in distributed systems as well as large-scale computing environments, further sheds light on the importance of scalable algorithms that are capable of handling complex search spaces efficiently [8]. In addition, it is also a well-known fact that stochastic search and evolutionary feature-selection approaches have been successfully applied to several machine learning tasks, some of whose typical examples are classification and prediction, demonstrating their broader applicability to challenging optimization problems [18]–[21]. Some other studies have explored algorithmic improvements and system-level optimization in various AI-driven applications, and these studies show that this reinforces the need for systematic benchmarking and performance analysis [22]–[25].

III. METHODOLOGY

A. Overall Workflow

The overall workflow is shown in Figure 1.

1) **Exhaustive Search (DFS)**: Exhaustive Search, also known as Depth-First Search or Backtracking, is an outstanding technique that can systematically explore all possible queen placements column by column and/or row by row whichever is the main algorithm. If a safe position is unavailable in a column, the algorithm backtracks to the previous one. This continues until it either finds a valid configuration or exhausts all options.

The key steps for exhaustive search are mentioned below. We start by accepting a standard size of the board N (10, 30, 50, 100, 200) from the user as input. We start from the first column and check each row for a safe spot, where the queen can be placed without attacking another queen. Whenever a safe spot is found, we place one queen in the column ensuring that it does not have any conflict with the existing queens. If no safe spot is found in the column for a queen, we then backtrack and find a suitable alternate position for the previous queen. This process is continued until all queens are placed or all possibilities are exhausted. We track memory and time usage throughout the process. Finally, we display the solution on the $N \times N$ board.

2) **Greedy Search (Hill Climbing)**: Greedy Search tries to minimize queen conflicts on a randomly initialized board by making the best local move. Since it can get stuck in local optima, two enhancements are used: Max Restarts (to try new random boards) and P_sideways (to allow lateral moves when no better state is found).

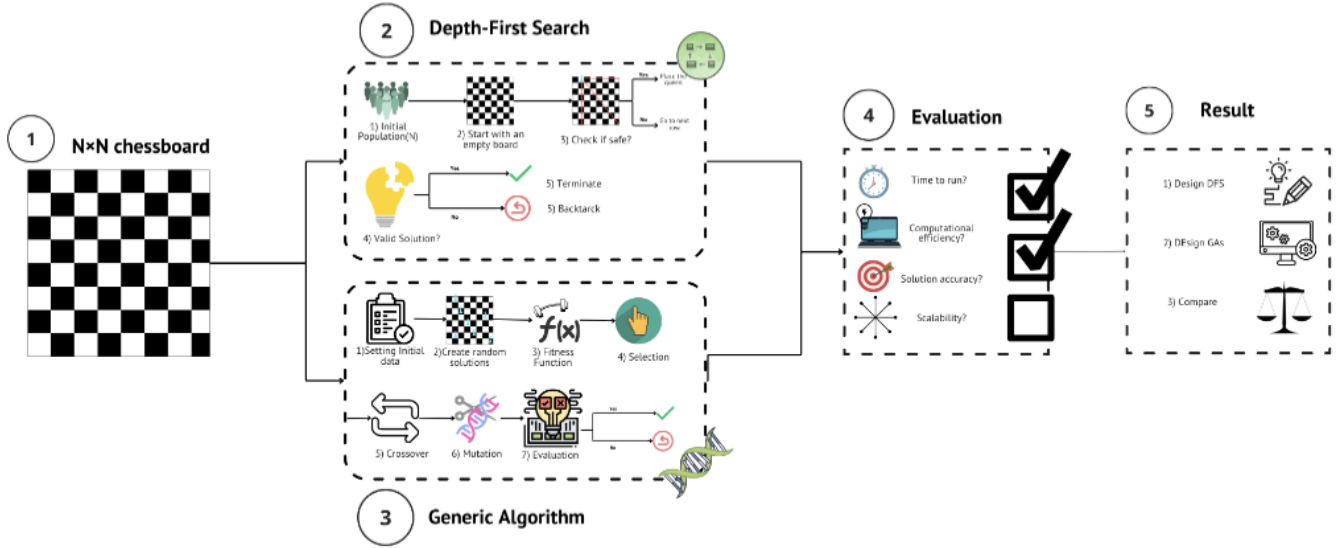


Fig. 1. Process Workflow for N-Queens Solutions for various techniques deployed in this study.

For greedy search technique using hill climbing, we applied the following steps. We start by accepting a standard size of the board N (10, 30, 50, 100, 200) from the user as input. This is followed by generating random boards and setting a maximum number of restarts, where each random board is generated by randomly placing a queen in each column at a random position. We then evaluate/count the number of conflicts using an $O(N^2)$ function, and then choose moves in the neighborhood (move one position up or down where possible) for each queen that reduce conflicts (minimum conflicts heuristic). We have used P_sideways to escape flat local optima. This method either finds a solution or we may have to restart with a new random board. We track memory and time usage throughout the process. Finally, we display the solution on the $N \times N$ board.

3) Simulated Annealing: Simulated Annealing is a meta-heuristic inspired by the annealing process in metallurgy. It accepts suboptimal moves early on, with the likelihood of doing so decreasing as the temperature cools. This helps the algorithm escape local minima and move toward better solutions. The key steps deployed for simulated annealing include the following steps.

We begin by setting initial and cooling temperatures to control acceptance of worse states. This is followed by generating a random board with one queen per column to set up an initial solution. We now start running the iterations such that at each iteration, we selected a random queen and move it to a new position. We then calculate the conflicts in the existing solution and decide whether to accept the new state based on a probability function (δ). With each iteration, the temperature is gradually lowered so that the probability of selecting bad moves are lowered over time and with each iteration. We repeat this process until a valid solution is found or the maximum number of steps is reached. Finally, we track

and report memory and time usage, and display the final board as a result.

4) Genetic Algorithm: The Genetic Algorithm, inspired by natural evolution, is a powerful technique for exploring large search spaces. It begins with a randomly generated population and applies operations like fitness evaluation, parent selection, crossover, and mutation to evolve better solutions over generations. To enhance performance and avoid local optima, the method incorporates elitism, local search for minimizing conflicts, and a stagnation check to reset the population when no progress is observed.

The key steps of the Genetic algorithm are shown here. We begin by setting the key parameters for Genetic algorithms, namely `pop_size`, `generations`, `p_crossover`, `p_mutation`, `k`, `stagnation`, `local_search_steps`. This is followed by initializing a random population based on N and `pop_size`. We then perform iterations through the predefined number of generations to evolve the candidate solutions. This is followed by selecting parents using fitness scores and the k -best individuals. We then applied elitism to carry top individuals into the next generation, which was then followed by performing crossover to produce new offsprings. A stagnation threshold is then used to reset population if stuck. We then applied mutation operator to introduce diversity in queen positions. This was followed by running local search to reduce remaining conflicts if any existed, especially for larger board sizes N . We keep track of memory and time usage throughout the process, and display the final board solution at the end of the algorithm.

B. Experimental Environment

As shown in Table II, all four algorithms—Exhaustive Search, Greedy Search, Simulated Annealing, and Genetic Algorithm—were implemented in Python and tested on the same hardware setup. Each was run three times on board sizes

TABLE I
CORE PARAMETERS FOR GENETIC ALGORITHM OPERATION

N	Pop_size	Generations	p_crossover	p_mutation	k	elitism	Stagnation	local_search_steps
10	40	100	0.85	0.18	3	2	10	2
30	80	160	0.93	0.14	4	2	15	2
50	100	200	0.8	0.1	3	2	50	1
100	600	1200	0.92	0.17	5	2	70	2
200	2000	1200	0.90	0.18	6	6	120	10

N = 10, 30, 50, 100, and 200 to evaluate performance in terms of time, memory, and success rate.

TABLE II
HARDWARE AND SOFTWARE SPECIFICATIONS

Type	Specification
CPU	Apple M1 Pro
RAM	16GB
Storage	512GB SSD
Operating System	macOS
Programming Language	Python3.10.5
IDE	Visual Studio Code

1) *Exhaustive Search*: There are no specific parameters that need to be considered in this algorithm(DFS). It will list all possibilities depending on the numbers selected to find the answer.

TABLE III
GREEDY SEARCH PARAMETERS

N	Max_restarts	P_sideways
10	20	0.3
30	20	0.3
50	20	0.3
100	50	0.4
200	50	0.4

2) *Greedy Search*: The Greedy Search (Hill Climbing) algorithm initializes queens randomly and moves each to reduce conflicts. To reduce the chance of getting trapped in local minima, two parameters were introduced Table III:

- Max_restarts: 20–50 — number of times the board is reinitialized.
- P_sideways: 0.2–0.5 — allows lateral moves when no improvement is found.

These help improve success rates, especially for larger boards.

TABLE IV
REQUIRED PARAMETERS IN SIMULATED ANNEALING

N	Max_restarts	P_sideways	Cooling
10	2,000	2.0	0.99
30	10,000	2.0	0.99
50	50,000	10.0	0.995
100	100,000	20.0	0.999
200	1,000,000	40.0	0.9995

3) *Simulated Annealing*: Simulated Annealing relies heavily on three parameters for its success, particularly on larger boards (see Table IV):

- Max_steps: 1,000–2,000,000

- Table _temp: 1.0–60.0
- Cooling: 0.9–0.9999

Lower values work for small N, while higher N requires slower cooling and more steps to effectively explore the search space.

4) *Genetic Algorithm*: The Genetic Algorithm uses a population-based search with evolutionary operators. Key parameters were tuned based on problem size (Table I):

- pop_size: 30–3000
- generations: 90–1500
- p_crossover: 0.80–0.99
- p_mutation: 0.10–0.30
- k: 2–8
- Elitism: 2–6 — carries top individuals to the next generation
- Stagnation threshold: 10–120 — resets the population if no progress is observed
- local_search_steps: 1–12 — reduces remaining conflicts post-mutation

These additions helped improve stability and success rates, particularly for larger N.

IV. EXPERIMENTAL ANALYSIS

In this section, experimental results and analysis of four algorithms trying to find the N-Queens problem were presented and were executed three times to ensure the consistency and reliability. Each algorithm is examined to show main performance metrics such as accuracy, time taken, memory usage, and to verify solution is found or not. Moreover, in certain instances, we have returned generations and steps taken to improve our understanding of the code.

A. Simulated Annealing results

Getting the initial temperature and rate of cooling correctly has a strong influence on the performance of algorithm. As shown in the Table V, When N is low ($30 \geq N \geq 10$), having low temperature and quick cooling(0.99) is highly recommended. Despite this, when N increases, there is a necessity to begin with high temperature and change it gradually. According to the results, the algorithm works well to solve the N-Queens problem from N=10 to N=100. When N goes up, the number of steps and the time duration will increase substantially. For instance, SA solved N=10 in just 0.0520 seconds with 1,903 steps, while N=100 required 71,439 steps and over 320 seconds to be completed. Moreover, memory usage remained steady between 2.45 and 4.49 KB. However, for N=200, the algorithm failed to find a solution

and regarding to key parameters such as 'Start_temp'=40.0, 'Max_steps'=1,000,00, and 'Cooling'=0.9995 more near to 1, but did not reach the point.

TABLE V
IMPACT OF N VALUES ON SIMULATED ANNEALING PERFORMANCE

N	Steps	Time(s)	Memory (KB)	Solution Found?	Notes
10	1093	0.0520	2.45	Yes	Quickly
30	6446	2.3253	2.85	Yes	Fast
50	29333	31.6541	3.32	Yes	Moderate time
100	71439	320.5094	4.49	Yes	Slower
200	-	-	-	-	Requires more steps and tuning finely

B. Genetic Algorithm

As we can see in the Table VI, the Genetic Algorithm was able to solve the N-Queens problem for N=10, 30, 50, and 100 successfully. In smaller numbers like 10 and 30, it found the solution by taking approximately between 0.0818 and 11.8087 seconds, and memory consumption of something between 22.39 and 68.66 KB. It must be considered that those parameters like generations, p_mutation, and also p_crossover and others have a positive impact on the result we get for each of the values. As N increased the number of generations, response time, and memory consumer raised more than in smaller ones. Fore example, for N=50 and N=100, respectively the time has gone drastically from 62.4965 seconds to 364.3041 and the memory exploitation was reached 1156.70 by increasing generation steps. Even with fine tuning some parameters to avoid getting stuck in local optima and jumping out of that situation, unfortunately, no answer was found for the N=200.

TABLE VI
TESTING GENETIC ALGORITHM WITH FIVE DISTINCT INPUT VALUES

N	Generation	Time(s)	Memory (KB)	Solution Found?	Notes
10	5	0.0818	22.39	Yes	Quickly to solve
30	14	11.8087	68.66	Yes	Solved
50	24	62.4965	114.89	Yes	Took time
100	48	364.3041	1156.70	yes	Time consuming
200	-	-	-	-	-

C. Performance Analysis

As shown in Figure 2, DFS performs well on small inputs, completing N=10 in 0.46 seconds. However, it's not evaluated for larger N due to impractical time and memory requirements. Greedy Search (GS) is fast for smaller boards (0.55s at N=10) but its runtime grows steeply, reaching 40.28 seconds at N=50. Simulated Annealing (SA) follows a similar early trend but scales better, solving N=100 in 320.51 seconds. Genetic Algorithm (GA), while capable of handling higher N values, shows a sharp increase in runtime, rising from 0.08 seconds (N=10) to 364.30 seconds (N=100).

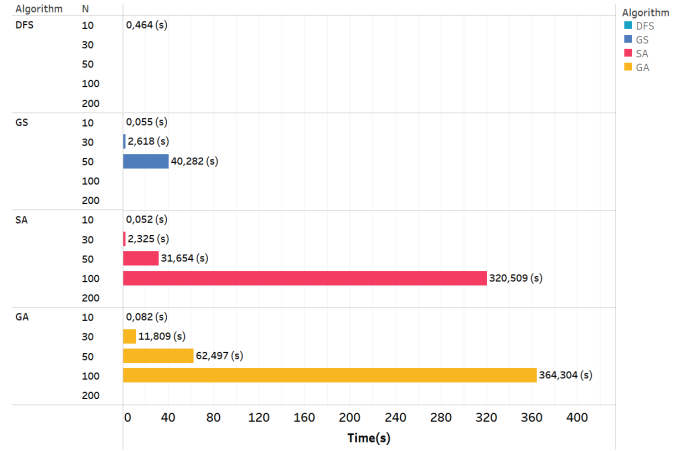


Fig. 2. Execution Time of N-Queens Solutions Across Varying Board Sizes

In terms of memory, GS and SA are the most efficient, using under 5 KB even at larger N. DFS, in contrast, consumes over 100 KB at N=10 alone and is not measured beyond that. GA, while effective for larger boards, requires significantly more memory—reaching over 1,100 KB at N=100—due to population and genetic data structures. Overall, SA offers the best trade-off between runtime and memory, while GA trades efficiency for scalability.

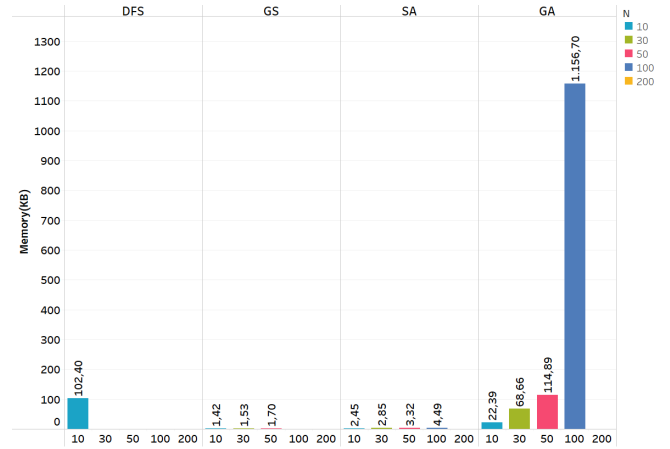


Fig. 3. Comparative Memory Usage of N-Queens Algorithms

V. DISCUSSION

Exhaustive search offers a fast memory response for small instances up to 15. Hill climbing is moderately efficient on medium-sized problems, and it is not enough to solve large instances. both Simulated Annealing (SA) and Genetic Algorithm (GA) were capable of solving the N-Queens problem up to boards with N=100. On the other side, it was clear that they ran differently in terms of pace, utilizing memory and solving larger problems. In cases where N was small ($50 \geq N \geq 10$), two algorithms such as SA and GS out performed more efficient and quicker compared to others. Additionally, Simulated Annealing paves the way for having better access to N=100

by exploiting low-memory and less time. Although, Genetic algorithm will be acted with overhead and consuming a lot of time and these problems become the obstacles of larger instances and no algorithm could find a solution with $N=200$, but Simulated Annealing with the selection of better tuning and Genetic Algorithm with a combination of methods will present the result exactly.

VI. CONCLUSION

The N-Queens problem continues to serve as a foundational benchmark for solving real-world optimization challenges. This study compared four algorithms—Exhaustive Search, Greedy Search, Simulated Annealing, and Genetic Algorithm—across board sizes ranging from $N=10$ to $N=200$, analyzing their performance in terms of time, memory usage, and scalability. Results showed that no single algorithm handles all scenarios equally well, where some are faster, others are more accurate, others are guaranteed to find solution, while some take less memory. Overall, the findings highlight the need for trade-offs between speed, accuracy, and resource consumption. Adaptive or hybrid approaches may offer better solutions for larger problem instances in future work.

DECLARATIONS

The authors used the generative AI tool ChatGPT (OpenAI, San Francisco, CA, USA) to improve the language and clarity of the manuscript. The authors reviewed and edited all content generated by the tool and take full responsibility for the final version of the manuscript.

REFERENCES

- [1] D. Carlberg, A. Raza, R. H. Ali, T. A. Khan, and S. Faiz, "Evaluating deterministic and stochastic strategies for large-scale n-queens solutions," in *2025 International Conference on Frontiers of Information Technology (FIT)*. IEEE, 2025, pp. 1–6.
- [2] E. M. M. Aly, M. M. Hamid, M. Imad, and R. H. Ali, "A comparative evaluation of search and metaheuristic algorithms for the n-queens problem: Scalability, efficiency, and success rates," in *2025 27th International Multitopic Conference (INMIC)*. IEEE, 2025, pp. 1–6.
- [3] L. Mikhailova, S. G. Abbas, M. Imad, and R. H. Ali, "Solving the n-queens puzzle: A performance study of classical and metaheuristic algorithms," in *2025 International Conference on Frontiers of Information Technology (FIT)*. IEEE, 2025, pp. 1–6.
- [4] D. Carlberg, A. Raza, R. H. Ali, T. A. Khan, and S. Faiz, "A comparative analysis of the algorithms dfs, hill climbing, simulated annealing, and genetic algorithm for solving the n-queens problem," in *2025 27th International Multitopic Conference (INMIC)*. IEEE, 2025, pp. 1–6.
- [5] O. K. Majeed, R. H. Ali, A. Z. Ijaz, N. Ali, U. Arshad, M. Imad, S. Nabi, J. Tahir, and M. Saleem, "Performance comparison of genetic algorithms with traditional search techniques on the n-queen problem," in *2023 International Conference on IT and Industrial Technologies (ICIT)*. IEEE, 2023, pp. 1–6.
- [6] A. Z. Ijaz, R. H. Ali, N. Ali, T. Laique, and T. A. Khan, "Solving graph coloring problem via graph neural network (gnn)," in *2022 17th International Conference on Emerging Technologies (ICET)*. IEEE, 2022, pp. 178–183.
- [7] U. Arshad, H. H. Ellahie, Z. ul Abideen, and R. H. Ali, "An enhanced genetic algorithm framework for efficient solutions to capacitated vehicle routing problems," in *2023 18th International Conference on Emerging Technologies (ICET)*. IEEE, 2023, pp. 141–146.
- [8] J. Zhang, Z. Ning, R. H. Ali, M. Waqas, S. Tu, and I. Ahmad, "A many-objective ensemble optimization algorithm for the edge cloud resource scheduling problem," *IEEE Transactions on Mobile Computing*, vol. 23, no. 2, pp. 1330–1346, 2023.
- [9] A. Maaz, S. Abbas, R. H. Ali, I. Ahmed, and T. A. Khan, "Vgg models in image captioning: Which architecture delivers better descriptions?" in *2024 18th International Conference on Open Source Systems and Technologies (ICOSST)*. IEEE, 2024, pp. 1–6.
- [10] M. Naeem, A. B. Siddique, R. H. Ali, U. Arshad, Z. ul Abideen, T. A. Khan, M. H. Shah, A. Z. Ijaz, and N. Ali, "Performance evaluation of popular deep neural networks for neural machine translation," in *2023 International Conference on Frontiers of Information Technology (FIT)*. IEEE, 2023, pp. 220–225.
- [11] A. W. Paracha, U. Arshad, R. H. Ali, Z. U. Abideen, M. H. Shah, T. A. Khan, A. Z. Ijaz, N. Ali, and A. B. Siddique, "Leveraging ai and nlp in chatbot development: An experimental study," in *2023 International Conference on Frontiers of Information Technology (FIT)*. IEEE, 2023, pp. 172–177.
- [12] Z. ul Abideen, T. A. Khan, R. H. Ali, N. Ali, M. M. Baig, and M. S. Ali, "Docontap: Ai-based disease diagnostic system and recommendation system," in *2022 17th International Conference on Emerging Technologies (ICET)*. IEEE, 2022, pp. 148–153.
- [13] I. Ahmad, G. Ahmed, S. A. A. Shah, and A. Ejaz, "A decade of big data literature: analysis of trends in light of bibliometrics," *The Journal of Supercomputing*, vol. 76, pp. 3555–3571, 2020.
- [14] D. Muhammad, I. Ahmad, M. I. Khalil, W. Khalil, and M. O. Ahmad, "A generalized deep learning approach to seismic activity prediction," *Applied Sciences*, vol. 13, no. 3, p. 1598, 2023.
- [15] D. Muhammad, I. Ahmed, K. Naveed, and M. Bendeche, "An explainable deep learning approach for stock market trend prediction," *Heliyon*, vol. 10, no. 21, 2024.
- [16] D. Muhammad, I. Ahmed, M. O. Ahmad, and M. Bendeche, "Randomized explainable machine learning models for efficient medical diagnosis," *IEEE Journal of Biomedical and Health Informatics*, vol. 29, no. 9, pp. 6474–6481, 2024.
- [17] D. Muhammad, I. Ahmed, K. Naveed, and M. Bendeche, "Explainable ai models for decoding emotional subtexts on social media," *Complexity*, vol. 2025, no. 1, p. 9258956, 2025.
- [18] A. Shabbir, R. H. Ali, M. Z. Shabbir, Z. U. Abideen, T. A. Khan, A. Z. Ijaz, N. Ali, M. Imad, and M. A. Bakar, "Genetic algorithm-based feature selection for accurate breast cancer classification," in *2023 International Conference on IT and Industrial Technologies (ICIT)*. IEEE, 2023, pp. 1–6.
- [19] M. Iftikhar, N. Ali, R. H. Ali, and A. Bais, "Classification of parkinson disease with feature selection using genetic algorithm," in *2023 IEEE Canadian Conference on Electrical and Computer Engineering (CCECE)*. IEEE, 2023, pp. 522–527.
- [20] F. Faridoun, R. H. Ali, Z. U. Abideen, N. Shahzadi, A. Z. Ijaz, U. Arshad, N. Ali, M. Imad, and S. Nabi, "Prediction of polycystic ovary syndrome using genetic algorithm-driven feature selection," in *2023 International Conference on IT and Industrial Technologies (ICIT)*. IEEE, 2023, pp. 1–6.
- [21] A. B. Siddique, M. A. Bakar, R. H. Ali, U. Arshad, N. Ali, Z. U. Abideen, T. A. Khan, A. Z. Ijaz, and M. Imad, "Studying the effects of feature selection approaches on machine learning techniques for mushroom classification problem," in *2023 International Conference on IT and Industrial Technologies (ICIT)*. IEEE, 2023, pp. 1–6.
- [22] C. K. Manjunatha, H. A. Khan, R. H. Ali, and U. R. Syed, "Efficientnetb0-based lightweight model for accurate and real-time american sign language recognition," in *2025 27th International Multitopic Conference (INMIC)*. IEEE, 2025, pp. 1–6.
- [23] R. Manchana, A. Gohar, M. Imad, and R. H. Ali, "Lightweight wildlife image classification using efficientnetv2-s with confidence-aware prediction," in *2025 27th International Multitopic Conference (INMIC)*. IEEE, 2025, pp. 1–6.
- [24] A. F. D. Miguel, A. Faisal, T. A. Khan, R. H. Ali, and S. Faiz, "Real-time american sign language recognition using efficientnetv2s: A transfer learning approach with webcam-based interface," in *2025 International Conference on Frontiers of Information Technology (FIT)*. IEEE, 2025, pp. 1–6.
- [25] T. Ahmed, A. Maaz, D. Mahmood, Z. ul Abideen, U. Arshad, and R. H. Ali, "The yolov8 edge: Harnessing custom datasets for superior real-time detection," in *2023 18th International Conference on Emerging Technologies (ICET)*. IEEE, 2023, pp. 38–43.